

TruckFlow Manager — Struttura futura del progetto

Scopo

Questo documento spiega come organizzeremo TruckFlow Manager dopo il completamento del `domain`.

Il progetto non deve diventare un insieme confuso di classi. Ogni cosa deve stare al proprio posto.

La regola principale sarà:

```
domain = regole del business
application = azioni del sistema
infrastructure = dettagli tecnici
web = API REST future
frontend = interfaccia grafica futura
```

1. Stato attuale del progetto

Al momento abbiamo costruito principalmente il `domain`.

Il domain contiene le regole fondamentali del business:

- ordini di trasporto;
- spedizioni;
- missioni operative;
- autisti;
- mezzi;
- combinazioni veicolo;
- carichi;
- ADR;
- percorsi;
- documenti;
- fatture;
- pagamenti;
- reclami;
- audit;
- notifiche;
- sostenibilità;
- configurazioni;
- report.

Il domain è stato progettato per essere Java puro.

Nel domain non devono entrare:

- database;
- Spring;
- controller REST;
- frontend;
- file TXT;
- repository tecnici;
- chiamate HTTP;
- Google Maps;
- ViaMichelin;
- email reali;
- login reale;
- JPA;
- JSON DTO.

Il domain deve rispondere alla domanda:

Quali sono le regole del business?

Esempi:

Una Shipment nasce solo da un TransportOrder accettato.
Un Driver senza CQC non può fare certi trasporti.
Una VehicleCombination deve essere compatibile con il CargoLoad.
Una Invoice ISSUED può diventare PAID.
Un GeneratedReport deve avere metriche se è GENERATED.

2. Prossimo macro-step: Application Layer

Dopo il domain, il prossimo step è l' `application layer` .

L'application layer risponde alla domanda:

Quali azioni può fare il sistema usando il domain?

Esempi:

Creare un ordine di trasporto.
Inviare un ordine.
Accettare un ordine.
Creare una spedizione da un ordine accettato.
Pianificare una spedizione.
Creare una missione operativa.

```
Emettere una fattura.  
Registrare un pagamento.  
Generare un report.
```

L'application layer non deve duplicare le regole del domain.

Il suo compito è coordinare il flusso.

Esempio:

```
CreateShipmentFromOrderUseCase
```

Questo use case farà:

1. Riceve il numero ordine.
2. Cerca l'ordine.
3. Verifica che l'ordine esista.
4. Usa il domain per creare la Shipment.
5. Salva la Shipment.
6. Restituisce il risultato.

La regola "una Shipment nasce solo da un TransportOrder accettato" resta nel domain.

3. Struttura generale che useremo

La struttura finale del progetto sarà questa:

```
src/main/java/it/gabriele/truckflow  
├── domain  
├── application  
├── infrastructure  
└── web
```

Per ora lavoreremo soprattutto su:

```
domain  
application  
infrastructure.memory
```

Il package `web` arriverà più avanti, quando introdurremo REST API e Spring Boot.

4. Package `domain`

Il package `domain` contiene tutto ciò che riguarda il business puro.

Esempi:

```
domain.order.TransportOrder
domain.shipment.Shipment
domain.driver.Driver
domain.fleet.VehicleCombination
domain.billing.Invoice
domain.reporting.GeneratedReport
```

Dentro il domain ci sono:

- entity;
- value object;
- enum;
- rules;
- validazioni;
- transizioni di stato;
- compatibilità tra oggetti.

Il domain non deve sapere dove vengono salvati gli oggetti.

Quindi il domain non deve sapere se useremo:

```
RAM
file TXT
file JSON
PostgreSQL
database H2
JPA
```

Il domain deve solo proteggere le regole.

5. Package `application`

Il package `application` contiene i casi d'uso del sistema.

La struttura iniziale sarà:

```
application
├─ order
```

```
├─ shipment
├─ mission
├─ billing
├─ reporting
└─ port
    └─ repository
```

Ogni sotto-package contiene use case relativi a una certa area.

5.1 application.order

Contiene i casi d'uso sugli ordini.

Esempi:

```
CreateTransportOrderUseCase
SubmitTransportOrderUseCase
AcceptTransportOrderUseCase
RejectTransportOrderUseCase
CancelTransportOrderUseCase
```

Significato:

```
CreateTransportOrderUseCase
```

Crea un nuovo ordine.

```
SubmitTransportOrderUseCase
```

Invia un ordine draft.

```
AcceptTransportOrderUseCase
```

Accetta un ordine submitted.

5.2 application.shipment

Contiene i casi d'uso sulle spedizioni.

Esempi:

```
CreateShipmentFromOrderUseCase
PlanShipmentUseCase
DispatchShipmentUseCase
MarkShipmentInTransitUseCase
DeliverShipmentUseCase
CancelShipmentUseCase
```

Significato:

```
CreateShipmentFromOrderUseCase
```

Crea una spedizione partendo da un ordine accettato.

```
PlanShipmentUseCase
```

Pianifica una spedizione.

5.3 application.mission

Contiene i casi d'uso sulle missioni operative.

Esempi:

```
CreateTransportMissionUseCase
DispatchTransportMissionUseCase
StartTransportMissionUseCase
CompleteTransportMissionUseCase
CancelTransportMissionUseCase
```

La missione operativa è il viaggio reale.

5.4 application.billing

Contiene i casi d'uso economici.

Esempi:

```
IssueInvoiceUseCase
RegisterPaymentUseCase
MarkInvoicePaidUseCase
CancelInvoiceUseCase
```

5.5 application.reporting

Contiene i casi d'uso sui report.

Esempi:

```
GenerateReportUseCase  
PublishReportUseCase  
ArchiveReportUseCase
```

6. Command

Per ogni use case useremo un oggetto chiamato `Command`.

Un command rappresenta i dati necessari per eseguire un'azione.

Esempio concettuale:

```
CreateTransportOrderCommand
```

Contiene i dati necessari per creare un ordine.

Il command non è una regola di business.

È solo l'input dell'azione.

Esempio:

```
orderNumber  
customerAccount  
cargoLoad  
pickupFacility  
deliveryFacility  
pickupTimeWindow  
deliveryTimeWindow  
serviceType  
quotedPrice  
notes
```

Perché usare i command?

Per evitare metodi enormi con troppi parametri.

Invece di avere:

```
create(orderNumber, customer, cargo, pickup, delivery, timeWindow, price,
notes...)
```

avremo concettualmente:

```
create(command)
```

È più leggibile e più ordinato.

7. Use Case

Un use case rappresenta un'azione del sistema.

Esempio:

```
CreateTransportOrderUseCase
```

Responsabilità:

1. Ricevere il command.
2. Controllare eventuali precondizioni applicative.
3. Chiamare il domain.
4. Salvare il risultato tramite repository.
5. Restituire l'oggetto creato o modificato.

Il use case non deve contenere regole profonde di business.

Esempio sbagliato:

```
Il use case riscrive tutte le regole di TransportOrder.
```

Esempio corretto:

```
Il use case chiama TransportOrder e lascia al domain la validazione.
```

8. Package `application.port.repository`

Questo package contiene le interfacce repository.

Le repository interface sono contratti.

Esempi:

```
TransportOrderRepository  
ShipmentRepository  
DriverRepository  
VehicleCombinationRepository  
InvoiceRepository  
GeneratedReportRepository
```

Una repository interface dice cosa serve all'application layer.

Esempio concettuale:

```
TransportOrderRepository
```

Responsabilità:

```
salvare un ordine  
cercare un ordine per numero  
controllare se un ordine esiste
```

La repository interface non dice se i dati stanno:

```
in RAM  
su file TXT  
su file JSON  
su PostgreSQL
```

Dice solo cosa deve essere possibile fare.

9. Package `infrastructure`

Il package `infrastructure` contiene i dettagli tecnici.

Qui entrano le implementazioni concrete.

Esempi futuri:

```
infrastructure.memory
infrastructure.file
infrastructure.persistence.jpa
infrastructure.external
infrastructure.notification
infrastructure.security
```

9.1 infrastructure.memory

È la prima infrastruttura che useremo.

Contiene repository che salvano i dati in RAM.

Esempi:

```
InMemoryTransportOrderRepository
InMemoryShipmentRepository
InMemoryDriverRepository
InMemoryVehicleCombinationRepository
```

Queste classi implementano le interfacce definite in:

```
application.port.repository
```

Esempio concettuale:

```
TransportOrderRepository
  ↓ implementata da
InMemoryTransportOrderRepository
```

Questo ci permette di testare i use case senza database.

9.2 infrastructure.file

Potrebbe arrivare in futuro.

Servirebbe per salvare dati su file, per esempio:

```
TXT
JSON
CSV
```

Non è obbligatorio, ma può essere utile come esercizio intermedio prima del database.

Esempio:

```
FileTransportOrderRepository
JsonShipmentRepository
```

9.3 `infrastructure.persistence.jpa`

Arriverà quando introdurremo il database.

Qui staranno:

```
JpaTransportOrderRepository
JpaShipmentRepository
JpaInvoiceRepository
```

Queste classi salveranno i dati su PostgreSQL o altro database.

In questa fase entreranno concetti come:

```
JPA
Entity tecniche
mapping
database tables
migration
transaction
```

Queste cose non devono entrare nel domain.

9.4 `infrastructure.external`

Qui staranno gli adapter verso servizi esterni.

Esempi:

```
ViaMichelinRouteCostEstimator
HereMapsRouteEstimator
GoogleMapsGeocodingAdapter
EmailNotificationSender
DocumentStorageAdapter
```

Il domain non chiama direttamente questi servizi.

Il flusso corretto sarà:

```
Use Case
  ↓
Application Port
  ↓
Infrastructure Adapter
  ↓
Servizio esterno
```

10. RAM, TXT o Database

Il sistema sarà progettato per poter cambiare il modo in cui salva i dati.

La logica sarà:

```
application usa interfacce
infrastructure fornisce implementazioni
```

Quindi potremo avere:

```
TransportOrderRepository
├── InMemoryTransportOrderRepository
├── FileTransportOrderRepository
└── JpaTransportOrderRepository
```

Il use case non cambia.

Oggi useremo:

```
InMemoryTransportOrderRepository
```

Domani potremo usare:

```
JpaTransportOrderRepository
```

senza riscrivere il domain e senza riscrivere la logica applicativa principale.

Questa è una versione più pulita e professionale della vecchia idea di `BusinessFactory`.

11. Factory o configurazione centrale

Per scegliere quale implementazione usare, in futuro potremo avere un punto centrale.

Esempio concettuale:

```
DEV mode → repository in RAM  
FILE mode → repository su file  
PROD mode → repository su database
```

All'inizio non useremo Spring.

Quindi potremo avere una factory semplice che crea gli oggetti necessari.

Più avanti, con Spring Boot, sarà Spring a fare questa scelta tramite configurazione e dependency injection.

12. Test

Avremo diversi tipi di test.

12.1 Domain test

Testano regole di business isolate.

Esempi:

```
Money non può essere negativo.  
Shipment non nasce da ordine non accettato.  
Driver senza ADR non può trasportare dangerous goods.  
Invoice issued può diventare paid.  
Report generated richiede metriche.
```

Questi test non usano repository.

12.2 Application test

Testano use case completi.

Esempio:

```
Creo un ordine tramite CreateTransportOrderUseCase.  
L'ordine viene salvato nella repository in RAM.  
Posso recuperarlo.  
Se provo a creare un ordine con numero duplicato, ricevo errore.
```

Questi test usano repository in-memory.

12.3 Infrastructure test futuri

Testeranno implementazioni tecniche.

Esempi:

```
JpaTransportOrderRepository salva davvero su database.  
FileTransportOrderRepository scrive e legge da file.  
ViaMichelinRouteCostEstimator interpreta correttamente la risposta esterna.
```

Questi test arriveranno più avanti.

13. Primo flusso da implementare

Il primo flusso reale sarà:

```
CreateTransportOrderUseCase
```

Obiettivo:

```
Creare un ordine di trasporto tramite application layer e salvarlo in RAM.
```

Classi coinvolte:

```
application.order.CreateTransportOrderCommand  
application.order.CreateTransportOrderUseCase
```

```
application.port.repository.TransportOrderRepository
infrastructure.memory.order.InMemoryTransportOrderRepository
```

Test:

```
CreateTransportOrderUseCaseTest
```

Risultato atteso:

```
Il sistema crea un TransportOrder.
Il TransportOrder viene salvato in RAM.
Il test recupera il TransportOrder.
Il test verifica che i dati siano corretti.
```

14. Flusso successivo

Dopo il primo use case, continueremo così:

```
SubmitTransportOrderUseCase
AcceptTransportOrderUseCase
CreateShipmentFromOrderUseCase
PlanShipmentUseCase
CreateTransportMissionUseCase
```

Questo ci permetterà di costruire il primo flusso completo:

```
CustomerAccount
  ↓
TransportOrder
  ↓
Shipment
  ↓
RoutePlan + Driver + VehicleCombination
  ↓
TransportMission
```

15. Cosa non fare subito

Non faremo subito:

```
Spring Boot
REST API
PostgreSQL
JPA
frontend
login reale
ViaMichelin reale
email reali
document storage reale
```

Motivo:

Prima dobbiamo far funzionare bene il cuore applicativo.

La sequenza corretta è:

```
Domain
↓
Application
↓
Infrastructure memory
↓
Test flussi
↓
Spring Boot / REST API
↓
Database
↓
Frontend
↓
Integrazioni esterne
```

16. Regola pratica: dove metto una classe?

Se è una regola di business

Va in:

```
domain
```

Esempi:

```
VehicleCombinationRules
DriverRules
```


ShipmentRules
BillingRules

Se è un'azione del sistema

Va in:

application

Esempi:

CreateTransportOrderUseCase
AcceptTransportOrderUseCase
CreateShipmentFromOrderUseCase
RegisterPaymentUseCase

Se è un contratto per salvare o recuperare dati

Va in:

application.port.repository

Esempi:

TransportOrderRepository
ShipmentRepository
InvoiceRepository

Se è il modo concreto in cui salvo dati

Va in:

infrastructure

Esempi:

```
InMemoryTransportOrderRepository
FileTransportOrderRepository
JpaTransportOrderRepository
```

Se è una API REST

Andrà in futuro in:

```
web
```

Esempi:

```
TransportOrderController
ShipmentController
InvoiceController
```

17. Regola principale da ricordare

Il domain non deve sapere niente del mondo esterno.

L'application layer usa il domain.

L'infrastructure serve l'application layer.

Il web chiama l'application layer.

Schema:

```
web
↓
application
↓
domain

infrastructure
↑
application port
```

18. Riassunto finale

Abbiamo finito il domain MVP.

Ora dobbiamo costruire l'application layer.

Il primo obiettivo è creare un ordine tramite use case e salvarlo in RAM.

La struttura che useremo sarà:

```
domain
  regole del business

application
  azioni del sistema

application.port
  contratti/interfacce

infrastructure.memory
  implementazioni temporanee in RAM

infrastructure.persistence
  database futuro

web
  API REST future
```

Questa struttura ci permette di partire semplice, ma senza costruire un progetto disordinato.

Oggi RAM.

Domani file.

Dopodomani database.

Il domain e i use case devono rimanere puliti e comprensibili.